МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
КАФЕДРА ИИТ

Лабораторная работа №5

По дисциплине «Современный платформы программирования»

Выполнил:
Студент 3 курса
группы ПО-12:
Ефимович С.Л.
Проверил:
Кулик А.Д.

Брест 2026

Цель: приобрести практические навыки разработки API и баз данных

Ход работы

Вариант 5. База данных Сборка компьютера

1. Реализовать базу данных из не менее 5 таблиц на заданную тематику. При реализации продумать типизацию полей и внешние ключи в таблицах;
2. Визуализировать разработанную БД с помощью схемы, на которой отображены все таблицы и связи между ними (пример, схема на рис. 1);
3. На языке Python с использованием SQLAlchemy реализовать подключение к БД;
4. Реализовать основные операции с данными (выборку, добавление, удаление, модификацию);
5. Для каждой реализованной операции с использованием FastAPI реализовать отдельный эндпойнт;

Код программы:

```
from fastapi import FastAPI, HTTPException
from sqlalchemy import create_engine, Column, Integer, String, ForeignKey
from sqlalchemy.orm import declarative_base, sessionmaker, relationship

DATABASE_URL = "sqlite:///./pc_builds.db"

engine = create_engine(DATABASE_URL, connect_args={"check_same_thread": False})
SessionLocal = sessionmaker(bind=engine)

Base = declarative_base()
app = FastAPI()

class User(Base):
   __tablename__ = "users"
   id = Column(Integer, primary_key=True)
   name = Column(String)

   builds = relationship("Build", back_populates="user")


class Build(Base):
   __tablename__ = "builds"
   id = Column(Integer, primary_key=True)
   name = Column(String)
   user_id = Column(Integer, ForeignKey("users.id"))

   user = relationship("User", back_populates="builds")
   components = relationship("BuildComponent", back_populates="build")
```

```
class ComponentType(Base):
    __tablename__ = "component_types"
    id = Column(Integer, primary_key=True)
    name = Column(String)

    components = relationship("Component", back_populates="type")


class Component(Base):
    __tablename__ = "components"
    id = Column(Integer, primary_key=True)
    name = Column(String)
    type_id = Column(Integer, ForeignKey("component_types.id"))

    type = relationship("ComponentType", back_populates="components")
    builds = relationship("BuildComponent", back_populates="component")


class BuildComponent(Base):
    __tablename__ = "build_components"
    id = Column(Integer, primary_key=True)
    build_id = Column(Integer, ForeignKey("builds.id"))
    component_id = Column(Integer, ForeignKey("components.id"))

    build = relationship("Build", back_populates="components")
    component = relationship("Component", back_populates="builds")


Base.metadata.create_all(bind=engine)

@app.post("/users/")
def create_user(name: str):
    db = SessionLocal()
    user = User(name=name)
    db.add(user)
    db.commit()
    return {"id": user.id, "name": user.name}


@app.get("/users/")
def get_users():
    db = SessionLocal()
    return db.query(User).all()


@app.post("/components/")
def create_component(name: str, type_id: int):
    db = SessionLocal()
    comp = Component(name=name, type_id=type_id)
    db.add(comp)
    db.commit()
    return comp


@app.post("/types/")
def create_type(name: str):
    db = SessionLocal()
    t = ComponentType(name=name)
```

```
    db.add(t)
    db.commit()
    return t


@app.post("/builds/")
def create_build(name: str, user_id: int):
    db = SessionLocal()
    build = Build(name=name, user_id=user_id)
    db.add(build)
    db.commit()
    return build


@app.post("/build/add_component/")
def add_component(build_id: int, component_id: int):
    db = SessionLocal()
    bc = BuildComponent(build_id=build_id, component_id=component_id)
    db.add(bc)
    db.commit()
    return {"message": "Component added to build"}


@app.get("/builds/")
def get_builds():
    db = SessionLocal()
    return db.query(Build).all()


@app.get("/builds/full/")
def get_full_builds():
    db = SessionLocal()
    builds = db.query(Build).all()

    result = []
    for build in builds:
        components = []
        for bc in build.components:
            components.append(
                {
                    "id": bc.component.id,
                    "name": bc.component.name,
                    "type": bc.component.type.name,
                }
            )

        result.append(
            {
                "id": build.id,
                "name": build.name,
                "user_id": build.user_id,
                "components": components,
            }
        )

    return result


@app.delete("/components/{component_id}")
```

```
def delete_component(component_id: int):
    db = SessionLocal()
    comp = db.query(Component).get(component_id)
    if not comp:
        raise HTTPException(status_code=404, detail="Not found")
    db.delete(comp)
    db.commit()
    return {"message": "Deleted"}
```

Результат работы программы:

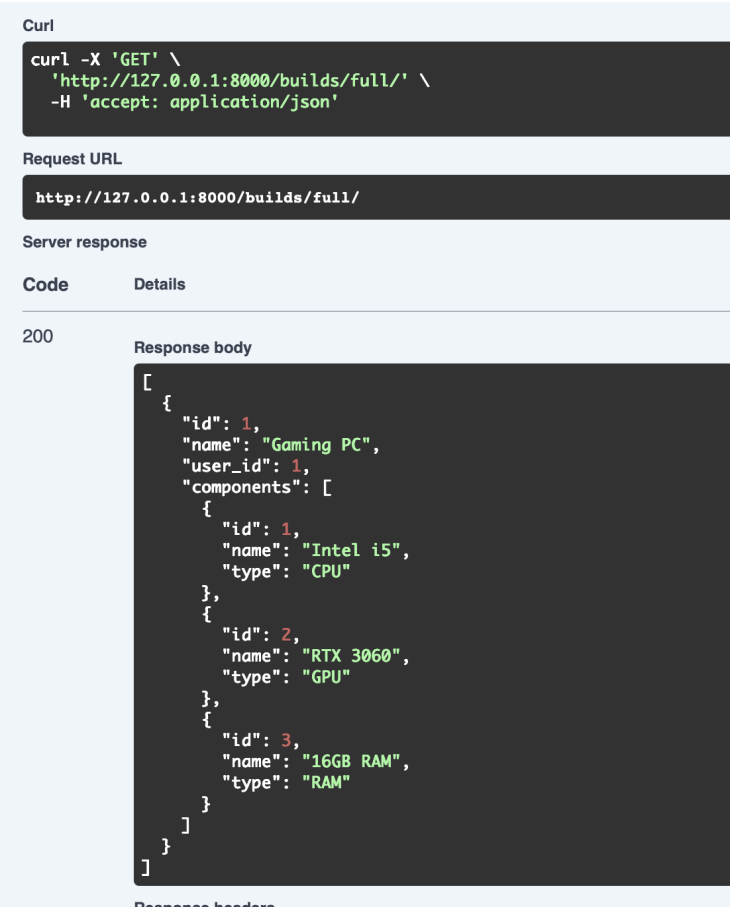

Рисунок 1. Результат выполнения программы

Вывод: приобрел практические навыки разработки API и баз данных